

What the Scrolls Didn't Say

A Real-World Microsoft Agent Framework Quest

A Few Heroes Walk Into a Tavern

Frederiek Vandepitte



in/fvandepitte



fvandepitte



Wouter Bauchart

in/wouterbau



wouterbau



The request

Our most **seasoned knights** are no longer forging new tools but instead **inspecting the work of apprentices**.

Endless reviews. Endless pipelines. Endless judgement.

Could an **intelligent** construct **assist** in judging the quality of this work, and restore balance to the guild?



Our response

*Yes, we can help restore balance.
But we would not build a simple tool.*

*We would build an intelligent construct-
One capable of assisting in the **judgement**
itself...*

*One that promised **great power!***

*One that was build on a **preview framework?***



Microsoft Agent Framework

*Created for .NET and Python
Based on Semantic Kernel and AutoGen*

*Uses **Microsoft.Extensions.AI**
Turn current AI clients into AI Agents
Reuse existing abstractions and functionalities*

*Provides **Agent and Workflow Orchestration**
Capabilities to
Desktop Applications
Web Applications
Background Tasks
...*



Microsoft Agent Framework

Out of the box support for

*Open Standards interoperability
(MCP, A2A, OpenAPI)*

Community, Enterprise and Custom Tools

Multimodal Content

Structured Output

RAG

YAML Declared Agent and Skill definitions

Memory

...



The history of Microsoft Agent Framework

May 2025



Build 2025

- *Agentic DevOps vision*
- *Separate systems announced*

The history of Microsoft Agent Framework

June 2025



Deep research tool

The history of Microsoft Agent Framework

October 2025



Microsoft agent announced

*Autogen and Semantic kernel in
maintenance mode*

Ignite 2025

- *Expanded agent capabilities*

The history of Microsoft Agent Framework

February 2026



Microsoft Agent Framework RC announced

The history of Microsoft Agent Framework

April 2026



Microsoft Agent Framework released

Our story



The September call to adventure



Wanted an evaluation tool

The September call to adventure



Wanted an evaluation tool

- *BUT with AI*

The September call to adventure



Wanted an evaluation tool

- *BUT with AI*
- *Agreed to use Microsoft Agent Framework*

The January Threshold

To move faster, we leaned on **GitHub Copilot**.

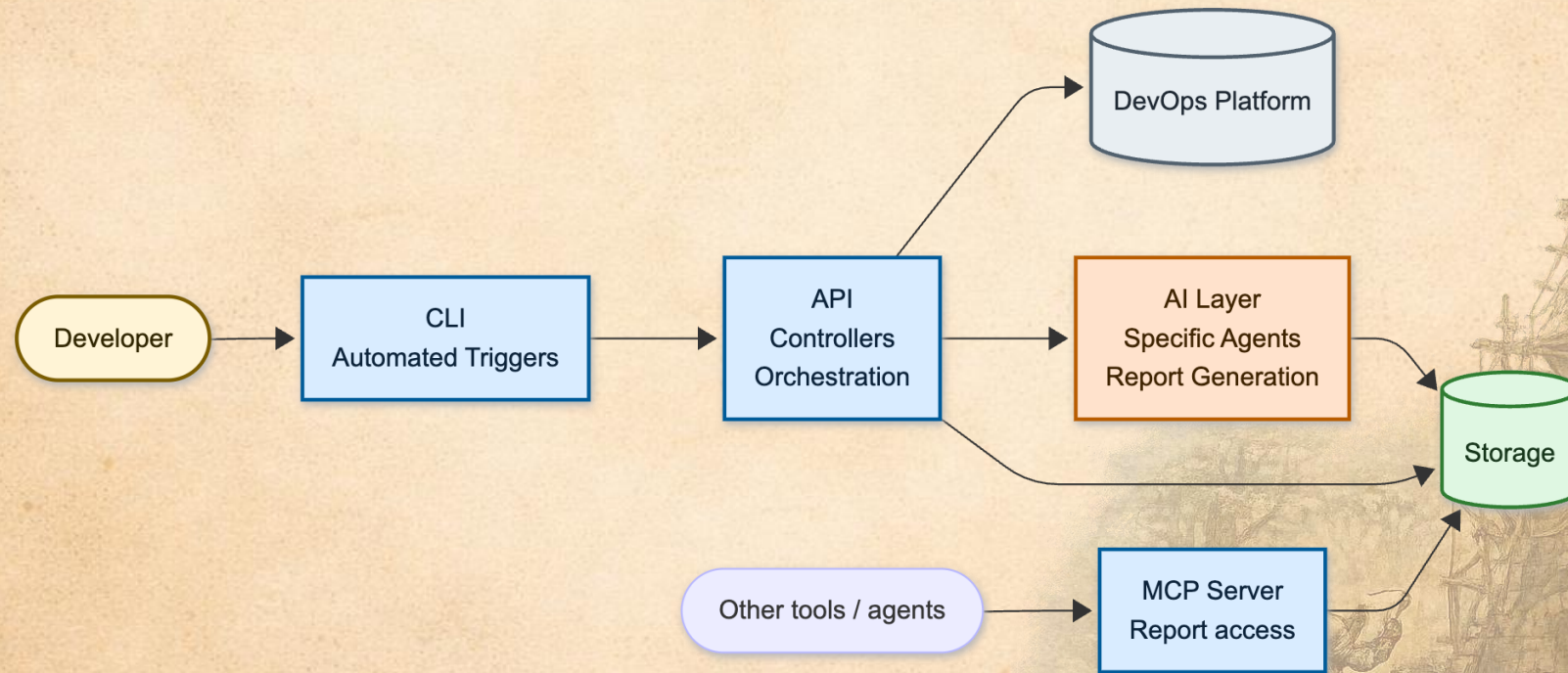
But with **preview technology**, **thin docs**, **shifting APIs**, and outdated examples, Copilot could **not reliably** guide us.

It helped with syntax.
It struggled with the unknown.



The January Threshold

We tried to build the full castle from day one...



The February mentorship



Back to the beginning

*Build the **smallest** useful **version** first*

***Validate** the approach*

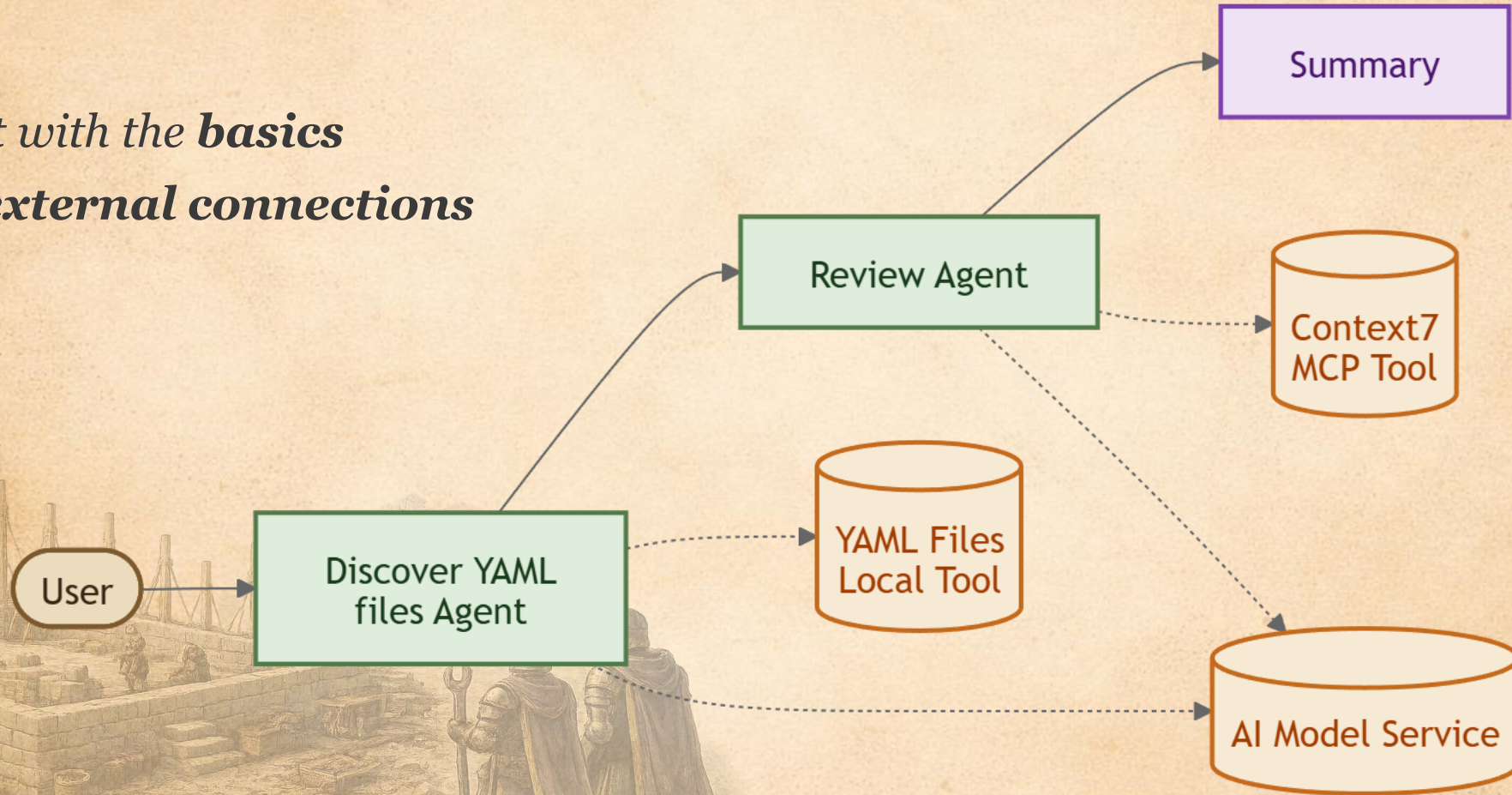
*Then **expand** with confidence.*

The February mentorship

Start small

Start with the basics

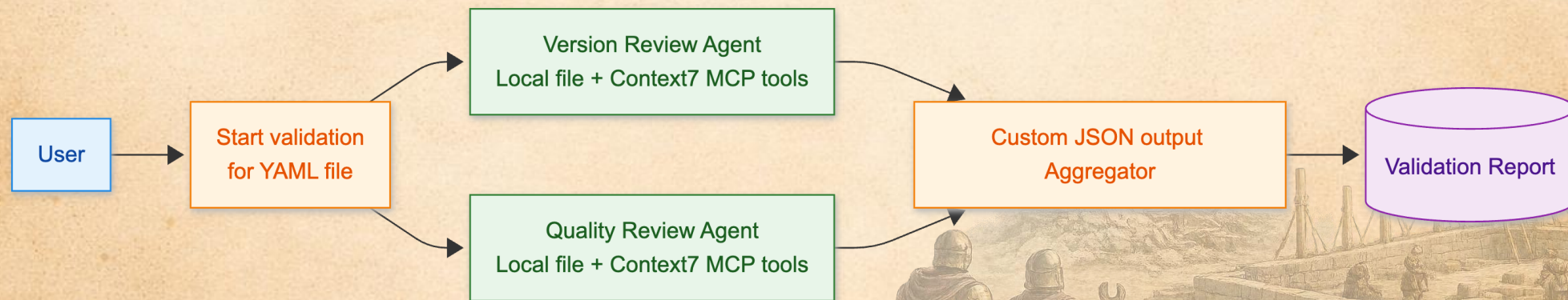
Add external connections



The February mentorship Expand with Confidence

Add **structure** to the **output** through **instructions**

Make sure we can handle **concurrent** processes



The March Transformation

*A **new hero** joins the party*

Tasked to review, improve and extend

*Trained for **Semantic Kernel***

Never to use that specific knowledge

Agent Framework is based on it

But provides a better implementation



Semantic Kernel Agent

```
#pragma warning disable SKEXP0070
var builder = Kernel.CreateBuilder();
builder.AddOllamaChatCompletion(model, new Uri(uri));
var kernel = builder.Build();

var agent = new ChatCompletionAgent()
{
    Name = "Story Agent",
    Kernel = kernel,
    Instructions = "You are a helpful assistant that creates funny creative stories."
};

await foreach (ChatMessageContent response in agent.InvokeAsync(
    "Describe a funny story of 3 lines about a dog barking at a cat"))
{
    Console.WriteLine($"{response.Content}");
}
```


Microsoft Agent Framework

```
var chatClient = new OllamaApiClient(uri, model);  
var agent = chatClient.AsAIAgent(  
    instructions: "You are a helpful assistant that creates funny creative stories.");  
var response = await agent.RunAsync(  
    "Describe a funny story of 3 lines about a dog barking at a cat");  
Console.WriteLine(response.Text);
```


Transforming to Agent Framework

```
#pragma warning disable SKEXP0110  
var afAgent = kernelAgent.AsAIAgent();  
var afResponse = await afAgent.RunAsync("Describe a funny cat in 2 lines.");  
Console.WriteLine(afResponse.Text);
```


The March Transformation Consolidation

A common project already existed

Used by all samples

But held only a couple of visual tools

Basics of Agent creation kept being duplicated

Configuring of

ChatClient

MCP tool

YAML File Retrieval

Agent Definitions and logic

Perfect to consolidate and reuse



The March Transformation Consolidation

```
public static class Tools
{
    [Description("Get the contents of a YAML file to investigate.")]
    public static string GetYamlContents([Description("The name of the YAML file.")] string fileName)
    {
        ...
    }
}
```


The March Transformation Consolidation

```
public static class ConfigurationRootExtensions
{
    public static async Task<McpClient> CreateContext7McpClientAsync(
        this IConfigurationRoot configuration)
    {
        ...
    }

    public static Task<IChatClient> CreateAzureOpenAiChatClientAsync(
        this IConfigurationRoot configuration)
    {
        ...
    }
}
```


The March Transformation Consolidation

```
public interface IAgentDefinition<out T>
{
    string AgentName { get; }
    string AgentDescription { get; }

    ChatOptions GetAgentChatOptionsWithProvidedTools(IList<AITool> tools);
    ChatResponseFormatJson GetAgentChatResponseFormatJson();
    T? GetOutputFromChatMessage(ChatMessage chatMessage);
}
```


The March Transformation Consolidation

```
<PackageReference Include="Azure.AI.OpenAI" Version="2.1.0" />  
<PackageReference Include="Microsoft.Agents.AI.Workflows" Version="1.0.0-preview.260128.1" />  
<PackageReference Include="Microsoft.Extensions.AI.OpenAI" Version="10.2.0-preview.1.26063.2" />  
<PackageReference Include="ModelContextProtocol.Core" Version="0.7.0-preview.1" />
```


The March Transformation

Structured Outputs

Structured outputs requested through instructions
Expected JSON being read as generic document

.NET Class could be configured instead
Ensuring

Output Structure

More Consistent Results

Additional Domain Logic



The March Transformation Structured Outputs

A simple Agent call:

```
AgentResponse<Story> response = await storyAgent.RunAsync<Story>(
    "Describe a funny cat in 2 lines");
```

When your Agent is part of a workflow:

```
var chatOptions = new ChatOptions()
{
    Instructions = AgentInstructions,
    ResponseFormat = ChatResponseFormat.ForJsonSchema<Story>(
        schemaName: "StoryResult",
        schemaDescription: "The generated story along with a synopsis of characters"),
    AllowMultipleToolCalls = true,
    Tools = tools
};
// Add ChatOptions to AI Agent creation
// Deserialize from JSON when retrieving responses from agent from results
```


The March Transformation Built-in Orchestration Patterns

Workflows constructed by custom edges and executors

Using AddFanOutEdge and AddFanInEdge

Too advanced for requirement

Agent Framework provides orchestration patterns

Less maintenance, better optimization

Built-in concurrent workflow sufficient



The March Transformation

Built-in Orchestration Patterns

Preparing custom fan-out to fan-in Orchestration:

// Executors are Implementations to handle messages within the workflow pipeline

// Agents can be converted to Executors

```
var workflow = new WorkflowBuilder(startExecutor)
    .AddFanOutEdge(
        startExecutor, [yamlVersionCheckingAgent, yamlBestPracticesCheckingAgent])
    .AddFanInBarrierEdge(
        [yamlVersionCheckingAgent, yamlBestPracticesCheckingAgent], aggregationExecutor)
    .WithoutOutputFrom(aggregationExecutor)
    .Build();
```

Using Built-In Concurrent Orchestration:

```
var workflow = AgentWorkflowBuilder.BuildConcurrent(
    [yamlVersionCheckingAgent, yamlBestPracticesCheckingAgent]);
```


The March Ordeal

*The time came to extend the samples
Making the agents available through a host
Which stirred the sleeping dragon*

*Updating to **RC version** became a **requirement**
Samples **broke faster** than we could trust them*

*Documentation **lagged** about the hosting features
Behavior and tools were to be learned by experience*



The March Ordeal

[Learn](#) > [Microsoft Agent Framework](#) >

Ask Learn

⋮

Directory Discovery

✦ Summarize this article for me

Choose a programming language

C# Python

DevUI can automatically discover agents and workflows from a directory structure. This enables you to organize multiple entities and launch them all with a single command.

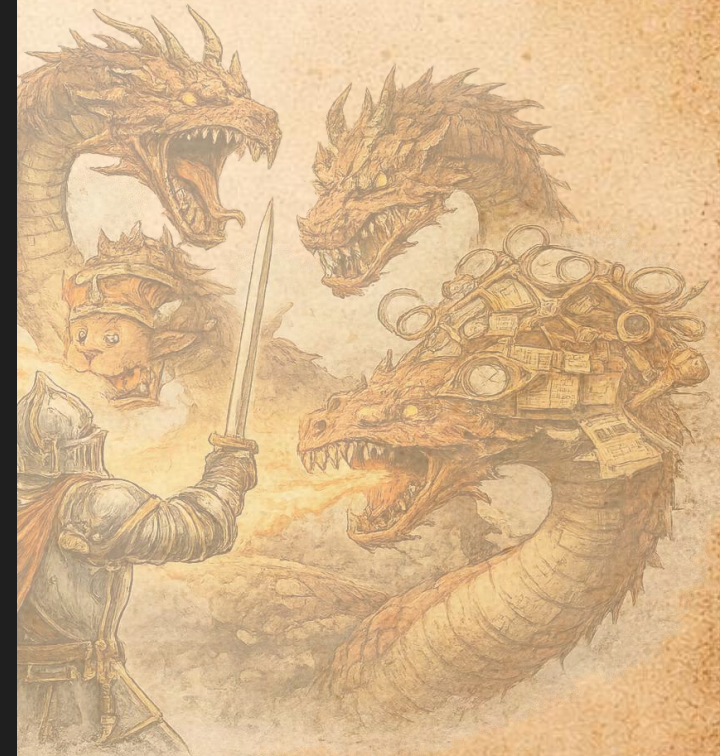
Coming Soon

DevUI documentation for C# is coming soon. Please check back later or refer to the Python documentation for conceptual guidance.

Next Steps

- [API Reference](#) - Learn about the OpenAI-compatible API
- [Tracing & Observability](#) - Debug your agents with traces

Last updated on 04/01/2026



The March Ordeal Updating to include hosting

Newest version at the time:

```
dotnet add package Microsoft.Agents.AI.Hosting --version 1.0.0-preview.260225.1
```

Minimal dependencies:

net10.0

Microsoft.Agents.AI ($\geq 1.0.0\text{-rc}2$)

Microsoft.Agents.AI.Abstractions ($\geq 1.0.0\text{-rc}2$)

Microsoft.Agents.AI.Workflows ($\geq 1.0.0\text{-rc}2$)

Microsoft.Extensions.AI ($\geq 10.3.0$)



The March Ordeal

Updating to include hosting

This meant updating the common project dependencies

```
<PackageReference Include="Microsoft.Agents.AI.Workflows" Version="1.0.0-rc3" />  
<PackageReference Include="Microsoft.Extensions.AI.OpenAI" Version="10.3.0" />  
<PackageReference Include="ModelContextProtocol.Core" Version="1.1.0" />
```

Which changed workflow component behavior

Breaking old samples beyond redemption

But Built-In Orchestration Patterns survived

Introducing build breaking errors like:

```
// Execute the workflow and stream the events as they happen. Before RC
```

```
var run = await InProcessExecution.StreamAsync(workflow, inputMessage);
```

```
// Execute the workflow and stream the events as they happen.
```

```
var run = await InProcessExecution.RunStreamingAsync(workflow, inputMessage);
```


The March Ordeal

Discovering DI and endpoint quirks

According to documentation

It all starts with a ChatClient

Then extension methods for Agents

```
// Register the chat client
IChatClient chatClient = new AzureOpenAIClient(
    new Uri(endpoint),
    new DefaultAzureCredential())
    .GetChatClient(deploymentName)
    .AsIChatClient();
builder.Services.AddSingleton(chatClient);

builder.AddOpenAIChatCompletions();

// Register an agent
var pirateAgent = builder.AddAIAgent("pirate", instructions: "You are a pirate. Speak like a pirate.");

var app = builder.Build();

app.MapOpenApi();
app.UseSwagger();
app.UseSwaggerUI();

// Expose the agent via OpenAI ChatCompletions protocol
app.MapOpenAIChatCompletions(pirateAgent);

app.Run();
```



The March Ordeal

Discovering DI and endpoint quirks

The truth is different when

Adding Telemetry

Using undocumented extension methods

Adding additional tools



```
var chatClient = (await builder.Configuration.CreateAzureOpenAiChatClientAsync())
    .AsBuilder()
    .UseOpenTelemetry()
    .Build();

// Clients are thread safe, this extension method registers it in DI
builder.Services.AddChatClient(chatClient);

// Context 7 MCP tool will be used by multiple agent definitions
await using var context7McpClient = await builder.Configuration.CreateContext7McpClientAsync();
var context7McpTools = await context7McpClient.ListToolsAsync();
IList<AITool> tools = [.. context7McpTools];

// Creating an agent, registering it also as a thread safe singleton in DI
builder.AddAIAgent(
    YamlVersionValidationAgentDefinition.AgentName,
    (serviceProvider, _) =>
    {
        var client = serviceProvider.GetRequiredService<IChatClient>();
        var agent = client.AsAIAgent(new ChatClientAgentOptions
        {
            Name = YamlVersionValidationAgentDefinition.AgentName,
            Description = YamlVersionValidationAgentDefinition.AgentDescription,
            ChatOptions = YamlVersionValidationAgentDefinition.GetAgentChatOptionsWithProvidedTools(tools)
        })
        .AsBuilder()
        .UseOpenTelemetry() // Enable OpenTelemetry for this agent.
        .Build();
        return agent;
    });
```


The March Ordeal

Discovering DI and endpoint quirks

*The truth is different when
Providing multiple Protocols*



```
// Discovers the agents from DI for OpenAI  
app.MapOpenAIResponses();  
app.MapOpenAIConversations();  
  
// Require to define per agent specifically for AG-UI  
app.MapAGUI("/validate-yaml/" +  
    YamlVersionValidationAgentDefinition.AgentName,  
    app.Services.GetKeyedService<AIAgent>(  
        YamlVersionValidationAgentDefinition.AgentName) ??  
        throw new InvalidOperationException());  
  
// Discovered through attributes on classes for MCP tools  
(Depends on configuration)  
app.MapMcp(pattern: "/mcp");
```


The March Ordeal

Adding test tooling

*DevUI is an additional application to include
Visualize and test agent endpoi*

Perfect platform to host test tooling

```
<PackageReference Include="CommunityToolkit.Aspire.Hosting.McpInspector"  
    Version="13.1.2-beta.518" />
```

But it didn't work out of the box...



The March Ordeal

Adding test tooling

Aspire was used to host the sample

Providing a safe environment to connect and test

Perfect platform to host test tooling

```
<PackageReference Include="CommunityToolkit.Aspire.Hosting.McpInspector"  
  Version="13.1.2-beta.518" />
```

But it didn't work out of the box...



The March Ordeal

Adding test tooling

 YamlValidation

Update now Aspire 13.2.4 is available. [Upgrade instructions](#)

Ignore

Console logs

mcp-inspector: Watching logs...

```
12
13 New StreamableHttp connection request
14 Query parameters: {"url":"https://localhost:7191/mcp","transportType":"streamable-http"}
15 Created StreamableHttp client transport
16 Client <=> Proxy sessionId: 372b4719-8c5a-48cf-bca1-5e9e92c7cd8c
17 stderr Proxy <=> Server sessionId: 0V7x0asFe9_evSvn0G6UVw
18 Received POST message for sessionId 372b4719-8c5a-48cf-bca1-5e9e92c7cd8c
19 Received GET message for sessionId 372b4719-8c5a-48cf-bca1-5e9e92c7cd8c
20 stderr node:internal/webstreams/readablestream:1068
21 stderr      throw new ERR_INVALID_STATE.TypeError('Controller is already closed');
22 stderr      ^
23 stderr
24 stderr TypeError [ERR_INVALID_STATE]: Invalid state: Controller is already closed
25 stderr   at ReadableStreamDefaultController.close (node:internal/webstreams/readablestream:1068:13)
26 stderr   at PassThrough.<anonymous> (file:///Users/wouterbauchart/.npm/_npx/7c622f3bacffc9df/node_modules/@modelcontextprotocol/inspector/server/build/index.js:196:28)
27 stderr   at PassThrough.emit (node:events:508:20)
28 stderr   at endReadableNT (node:internal/stream_readable:1729:12)
29 stderr   at process.processTicksAndRejections (node:internal/process/task_queue:90:21) {
30 stderr     code: 'ERR_INVALID_STATE'
31 stderr   }
32 stderr
33 stderr Node.js v25.6.1
```

MCP Inspector v0.17.2

Transport Type

Streamable HTTP

URL

https://localhost:7191/mcp

Connection Type

Via Proxy

Server Entry

Servers File

> Authentication

> Configuration

> Connect

Connection Error - Check if your MCP server is running and proxy token is correct



The March Ordeal Adding test tooling

Solution discovered through SKILL.md from online repository...

<https://lobehub.com/skills/jerrynixon-github-copilot-sql-dab-azure-mcp-aspire-mcp-inspector>

*Aspire Extension did not use
latest version of MCP Inspector*

Package

XML

```
<PackageReference Include="CommunityToolkit.Aspire.Hosting.McpInspector" Version="13.1.1" />
```

Canonical Program.cs Pattern

C#

```
var mcpInspector = builder
    .AddMcpInspector("mcp-inspector", options =>
    {
        options.InspectorVersion = "0.20.0"; // always pin - default version has known StreamableHTTP bugs
    })
    .WithMcpServer(dabServer, transportType: McpTransportType.StreamableHttp)
    .WithParentRelationship(dabServer) // groups under the MCP server in the dashboard
    .WithEnvironment("DANGEROUSLY_OMIT_AUTH", "true") // removes token prompt for local dev
    .WaitFor(dabServer);
```

Installation Method

View Details >

I'm an Agent

I'm a Human

Send this prompt to your agent to install the skill

Agent prompt

```
Curl https://lobehub.com/skills/jerrynixon-github-copilot-sql-dab-azure-mcp-aspire-mcp-inspector/skill.md, then follow the instructions to set up LobeHub Skills Marketplace and install the skill. Once installed, read the SKILL.md file in the installed directory and follow its instructions to complete the task.
```

Copy Prompt

Table of contents

- Documentation references
- Package
- Canonical Program.cs Pattern
- Transport Type

The March Ordeal

Adding test tooling

MCP Inspector v0.20.0

Transport Type

Streamable HTTP

URL

https://localhost:7191/mcp

Connection Type

Via Proxy

Server EntryServers File

> Authentication

> Configuration

ReconnectDisconnect

Connected

YamlValidation.Api

Version: 1.0.0.0

Logging Level

debug

System

ResourcesPromptsToolsTasksAppsPingSamplingElicitationsRootsAuth

Tools

List Tools

Clear

Select a tool

Select a tool from the list to view its details and run it

History

Clear

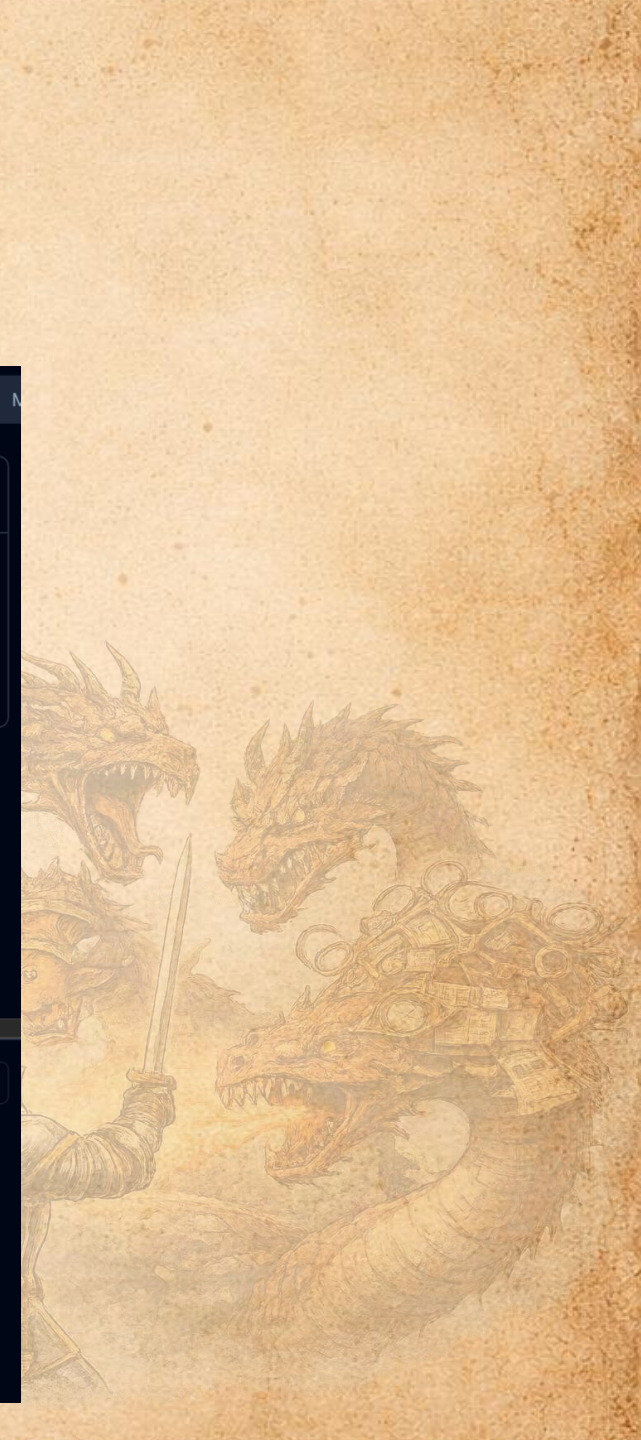
2. logging/setLevel

1. initialize

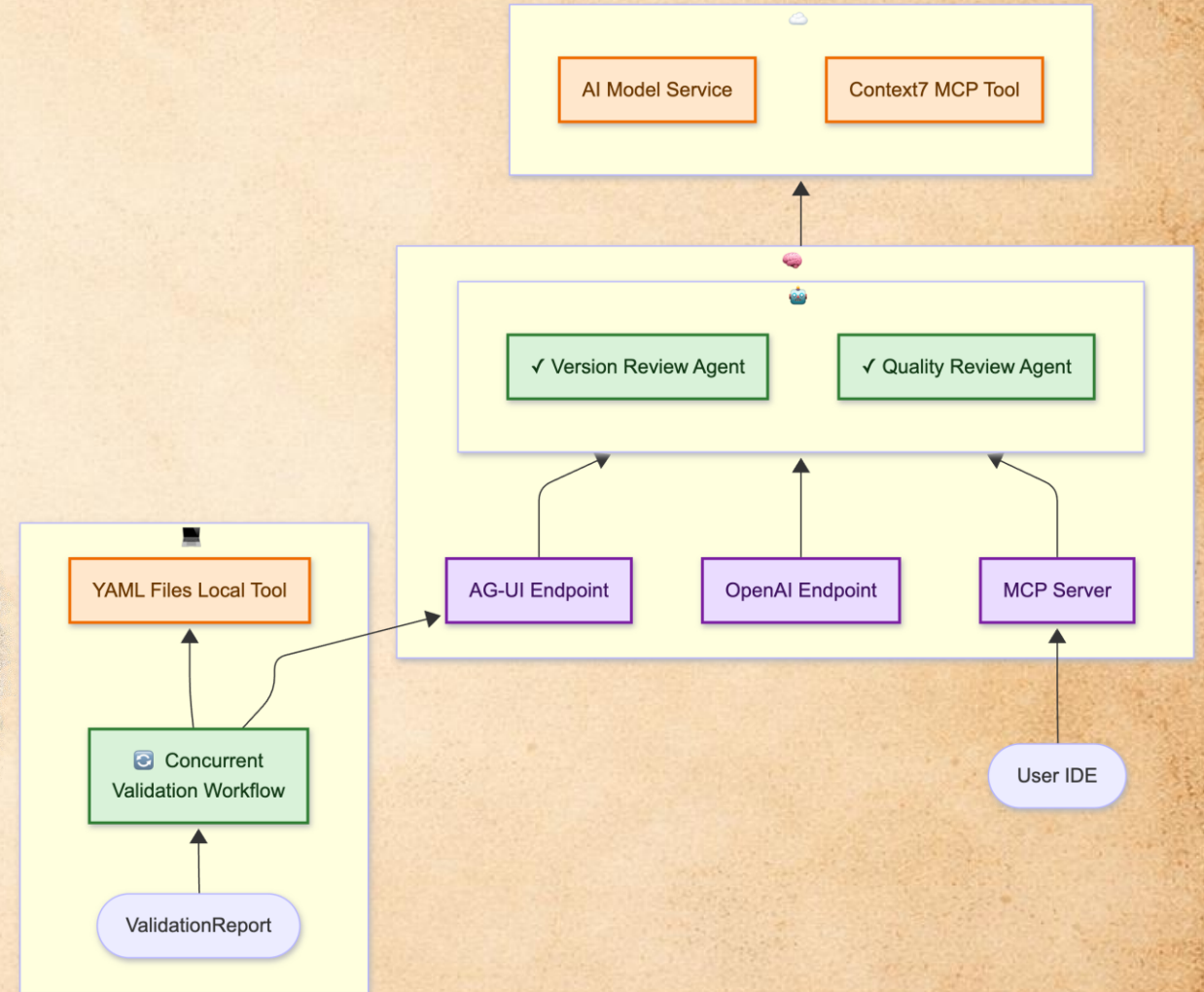
Server Notifications

Clear

No notifications yet



The March Reward



The April Resolution

Provided demo to customer

*Shared **same experiences** as their
development team*

*Proposal for **cooperation** accepted*



Lessons learned



Keep a continuous eyes for updates

Follow closely

All Communication Channels

Expect breaking changes

Look beyond official documentation

Embrace Twists, Turns and Chaos

Start small with Agentic Systems

Create Confidence

Learn the differences

Build up Experience

*, things were too different to what we
were used to*

Enquiries?

Thank you...

